

GitHub Copilot Beginner-Workshop

Zusammenfassung für Teilnehmende · 13. Juli 2026 · Remote · Cluster Reply, Agentic DevOps

Danke, dass du dabei warst. Dieses Dokument fasst den Workshop-Tag ausführlich zusammen – als Nachschlagewerk für die kommenden Wochen. Es gehört zu einem Materialpaket: Das Übungs-Repository bleibt für dich verfügbar, dazu kommen drei Begleitblätter – der **Quickstart** (CLI in 5 Minuten), das **Cheat-Sheet** (alle Befehle auf zwei Seiten) und die **Referenz** (Billing, Governance, IntelliJ). Alle Befehle hier funktionieren unter macOS, Linux und Windows (PowerShell, CMD, GitBash) identisch. Kopieren ausdrücklich erwünscht.

Die vier Ziele des Tages

- **Copilot verstehen:** wie es arbeitet, wo Stärken und Grenzen liegen – realistisch statt Hype.
- **Wirksam prompten:** Kontext steuern und Prompts so formulieren, dass die Vorschläge treffen.
- **Kosten im Griff:** AI Credits und Usage-Based Billing verstehen, Modelle bewusst wählen.
- **Die CLI als Zuhause:** vom ersten Login bis zur delegierten Aufgabe, alles im Terminal.

Modul 1 · Copilot verstehen

Vom Autocomplete zur Agenten-Plattform

2021 war Copilot ein Autovervollständiger („Tab-Tab“), 2023 kam der Chat dazu. Der eigentliche Sprung kam 2025 mit Agenten: Copilot erledigt mehrschrittige Aufgaben – lokal im Editor, im Terminal und asynchron in der Cloud. 2026 ist es eine Plattform: Du wählst das Modell, erweiterst über MCP-Server, und bezahlt wird nach tatsächlichem Verbrauch. Wer Copilot 2023 ausprobiert und abgehakt hat, sollte neu hinschauen – die Fähigkeiten von heute gab es vor 18 Monaten schlicht nicht.

Was passiert, wenn du einen Prompt absetzt?

Drei Stationen: Dein Client (bei uns die CLI) sammelt den Kontext. Der Copilot-Dienst baut daraus die Anfrage, prüft die Policies deiner Organisation und filtert. Das KI-Modell erzeugt den Vorschlag. Und dann kommt der wichtigste Pfeil des Diagramms: der zurück zu dir. **Nichts wird ausgeführt, bevor du es freigegeben hast.**

Trust & Datenschutz

Für Business- und Enterprise-Pläne gilt: Prompts und Code werden nicht zum Modelltraining verwendet; die Übertragung ist verschlüsselt, und Org-Policies steuern Features, Modelle und Datenfluss. Die belastbare, zitierfähige Quelle für deine Compliance-Abteilung ist das GitHub Copilot Trust Center.

Das Sicherheitsnetz zwischen Prompt und Antwort

Im Copilot-Proxy arbeiten mehrere Filter: Ein Intent-Classifizier prüft, ob die Anfrage code-bezogen ist. PII- und Toxicity-Filter fangen Persönliches und Problematisches ab. Die Duplicate Detection blockt Vorschläge, die öffentlichem Code stark ähneln – wichtig für die Lizenzfrage, standardmäßig aktiv und per Org-Policy steuerbar. Zusätzlich erkennen KI-Filter unsichere Muster (etwa SQL-Injection) direkt im Vorschlag. Die ehrliche Grenze: Downstream-Schwachstellen sieht kein Filter. Deshalb gilt: Copilot-Code wird behandelt wie Code von Kolleg:innen – Linting, Scanning und Review bleiben Pflicht.

Vier Arten, mit Copilot zu arbeiten

Completions (Inline-Vorschläge in der IDE), **Chat** (Fragen und Erklärungen), die **CLI** (interaktiv und agentisch im Terminal – unser Zuhause an diesem Tag) und der **Coding Agent** (Task rein, Draft-PR raus, asynchron in der Cloud). Für CLI-Teams wichtig: Praktisch jede Interaktion ist Modell-Arbeit – Kontext und Modellwahl zählen doppelt (mehr dazu in Modul 3).

Die Feature-Landkarte 2026 (alles General Availability)

Completions & Next Edit Suggestions, Chat (Ask & Edit), Copilot Spaces, Custom Instructions & AGENTS.md, Agent Mode & Plan Mode, die Copilot CLI, Cloud/Coding Agent mit parallelen Sessions, Custom Agents & Skills, Code Review & PR-Summaries, Model Picker, MCP-Server, Agent HQ/Mission Control, und seit Juni die GitHub Copilot App für den Desktop. Merke: Der GA-Status ändert sich laufend – die Quelle der Wahrheit ist der Changelog.

Sechs Meldungen aus drei Wochen: Das Tempo ist die Botschaft

17.06	GitHub Copilot App GA – der Desktop für Agent-Sessions
23.06	CLI: Tab-UI (Issues · PRs · Gists) jetzt Standard
25.06	Code Review nutzt CLI-Tools – rund 20 % günstiger
30.06	Claude Sonnet 5 & Opus 4.8 Fast im Model-Picker
01.07	Credit-Pools pro Cost Center (zunächst REST-API)
07.07	CLI v1.0.69: Sandbox-Feinschliff, /plugins-Dashboard

Diese Liste altert schnell – sie bildet den Stand vom 13.07.2026 ab. Deine Fünf-Minuten-Routine: einmal pro Woche github.blog/changelog.

Sprachen

Die Qualität folgt der Menge an öffentlichem Trainingscode: Python, JavaScript/TypeScript, Java, C# und Go sind sehr stark – euer Java-Stack ist Heimspiel. Auch SQL, Shell/PowerShell, Terraform, YAML und Markdown funktionieren gut im Alltag. Nischensprachen brauchen mehr Kontext und Beispiele.

Stärken und Grenzen – der Erwartungsabgleich

Copilot glänzt bei Boilerplate, Tests, dem Erklären fremden Codes, kleinen Refactorings und dem Regex, den niemand auswendig kann. Vorsicht bei Halluzinationen (plausibel klingende, aber falsche APIs – systembedingt, kein Bug), veralteten Mustern aus Trainingsdaten und allem, was dein Geschäftswissen voraussetzt. Der Merksatz des Tages: **Copilot ist Assistent, nicht Autopilot** – jedes Ergebnis wird gelesen, verstanden und gereviewt.

Sechs Alltagsmomente mit der CLI

- **Fremdes Repo verstehen:** „Erklär mir Struktur und Einstiegspunkte dieses Projekts.“
- **Bug eingrenzen:** Stacktrace in den Prompt, Hypothesen zurückbekommen.
- **Testlücke schließen:** Tests schreiben und ausführen lassen.
- **Shell statt Suchmaschine:** der Einzeiler, für den früher ein Browser-Tab aufging.
- **Commit-Message** aus den staged Änderungen, in Sekunden reviewfähig.
- **Doku-Absatz:** den README-Abschnitt ergänzen, der sonst liegen bleibt.

Modul 2 · Prompt Engineering

Wie ein LLM „denkt“

Dein Prompt wird in Tokens zerlegt; das Modell sagt Token für Token das wahrscheinlichste nächste Stück voraus – keine Datenbank, kein Nachschlagen, nur Wahrscheinlichkeiten. Daraus folgen drei Dinge: **Plausibel heißt nicht korrekt** (Halluzinationen sind eingebaut). **Das Wissen ist eingefroren** – Aktuelles gehört in den Prompt oder Kontext. Und es ist ein **Muster-Vervollständiger** – gute Beispiele zeigen wirkt stärker als beschreiben. Dazu das Kontextfenster: Prompt, Dateien und Verlauf teilen sich ein endliches Budget. Läuft es über, fällt der Anfang raus; in der CLI hilft /compact, und Relevanz schlägt Menge.

Was die CLI als Kontext sieht

Das freigegebene Arbeitsverzeichnis (sie liest und durchsucht selbst), Dateien und Pfade, die du nennst, den Session-Verlauf, deine Dauerregeln aus AGENTS.md, und angemeldet auch Repos, Issues und PRs auf GitHub. **Deine fünf Hebel:** im richtigen Ordner starten (das Verzeichnis ist der Kontext), Dateipfade explizit nennen, /context prüfen (zeigt dir die Auslastung des Kontextfensters), /compact bei langen Sessions, neue Session für neue Themen.

Fünf Techniken für bessere Vorschläge

Präzise statt vage (ein Satz Ziel plus drei Randbedingungen), **Beispiele geben** (Few-Shot: 1–2 Muster im Team-Stil), **Constraints setzen** (Sprache, Framework, No-Gos), **in Schritte zerlegen** (große Aufgaben = hohe Fehlerrate) und **iterieren** (Nachschärfen ist der normale Arbeitsmodus, kein Scheitern).

Vorher / Nachher

```
Vorher: "mach eine Funktion fuer Daten"

Nachher: "Schreibe eine TypeScript-Funktion
parseIsoDate(input: string): Date | null.
Ungueltige Eingaben -> null, keine Exceptions.
Keine externen Libraries. JSDoc + 3 Beispiel-Aufrufe."
```

Ein Satz Ziel + drei Randbedingungen – mehr braucht es meist nicht.

Das GitHub-Playbook: sechs Best Practices

Aus dem offiziellen Prompt-Engineering-Training: **1 Klare Instruktionen** (Persona, Format, Länge; Prompt-Teile mit Trennern strukturieren). **2 Referenzen mitgeben** (Quelltext oder Doku anhängen, senkt selbstbewusst-falsche Antworten). **3 Komplexes zerlegen**. **4 Denkzeit geben**: erst den Lösungsweg erklären lassen, dann den Code; die Antworten werden messbar besser. **5 Externe Tools nutzen**: Tests, Linter und Compiler prüfen zuverlässiger als das Modell selbst. **6 Systematisch testen**: Prompt-Änderungen wie Code-Änderungen behandeln.

Wenn der Vorschlag nicht passt: Symptom → Gegenmittel

zu generisch	Dateipfad nennen, Ziel schärfen – ein Satz Ziel + drei Randbedingungen.
erfundene API	Referenz anhängen: „Nutze ausschließlich Methoden aus dieser Datei/Doku.“
falscher Stil	einmalig per Few-Shot korrigieren, dauerhaft über AGENTS.md.
ufert aus / bricht ab	Aufgabe zerlegen; lange Session? /compact oder neue Session.
ignoriert Vorgaben	/context prüfen: Sieht die CLI, wovon du redest? Regeln fixieren.

Die Meta-Regel

Immer in dieser Reihenfolge: **1. Kontext prüfen · 2. Prompt schärfen · 3. erst dann das Modell wechseln**.
Neun von zehn schlechten Antworten sind ein Kontext- oder Prompt-Problem, kein Modell-Problem.

AGENTS.md – Team-Standards automatisch

Eine Markdown-Datei im Repo-Root, und jede CLI-Session und jeder Agent-Lauf startet mit euren Regeln – Stack, Konventionen, Test-Vorgaben, Do's & Don'ts. Der unterschätzte Vorteil: Die Datei liegt im Repo, läuft also durch euren Review-Prozess, Governance inklusive. Kurz halten: zehn präzise Zeilen schlagen drei Seiten Prosa, denn alles davon belegt Kontextfenster. Verwandt: `.github/copilot-instructions.md` deckt Chat und Coding Agent ab; beide ergänzen sich, nicht duplizieren.

```
# Projekt-Kontext
- Java 21, Maven, JUnit 5 + AssertJ
- Guard Clauses, Result statt Exceptions im Happy Path
- Keine Lombok-Annotationen
- Commit-Messages: Conventional Commits, englisch
- Vor jedem Commit: mvn -q test
```

Modul 3 · Usage-Based Billing & Modellwahl

Seit dem 01.06.2026: AI Credits statt Premium Requests

GitHub rechnet Copilot nutzungsbasiert ab: Beahlt wird die tatsächliche Modell-Arbeit in Tokens, zu Modell-API-Preisen. 1 Credit = 0,01 USD; jeder Plan bringt ein Monatsguthaben mit (Business 19 \$, Enterprise 39 \$ pro Lizenz), das bei Business/Enterprise auf Org-Ebene gepoolt wird. Wichtig: Es gibt **kein Fallback-Modell mehr** – ist das Guthaben aufgebraucht und kein Budget freigegeben, steht die Arbeit. Die drei Token-Arten: **Input** (was du sendest, wächst mit großen Dateien), **Output** (Antwort inkl. Tool-Nutzung – pro Token am teuersten), **Cache** (wiederverwendeter Kontext, am günstigsten; saubere Sessions sparen real Geld).

Aktionszeitraum – wichtig für die Budgetplanung: Bestandskunden mit Business/Enterprise erhalten vom **01.06. bis 01.09.2026** ein deutlich erhöhtes inkludiertes Credit-Kontingent. Danach fällt es auf die Standardwerte zurück. Verbrauchszahlen aus Juni bis August sind also nicht repräsentativ – Budgets und Alerts jetzt aufsetzen, nicht erst im September. Oberhalb der Base Credits greift zusätzlich ein variables Flex-Allotment, das GitHub automatisch anpassen kann.

Was kostet was?

- **Flatrate (keine Credits):** Code Completions & Next Edit Suggestions – unbegrenzt in bezahlten Plänen.
- **Credits:** Chat & Edits · Agent Mode · Copilot CLI · Coding Agent · Spark & Spaces.
- **Credits + Actions-Minuten:** Copilot Code Review und der Coding Agent (agentische Infrastruktur).

Merksatz: Tippen ist Flatrate, Denken kostet Tokens. Ein abgelehnter Completion-Vorschlag kostet nichts; eine verworfene Chat-Antwort schon – die Tokens sind geflossen. /usage zeigt den Verbrauch der laufenden Session.

Welches Modell wofür?

Der **Auto-Modus** ist der richtige Default: Er routet seit Juni aufgabenbasiert und meist kosteneffizient. Gezielt eingreifen lohnt in zwei Richtungen: **Frontier-Modelle** (z. B. Claude Sonnet 5/Opus, GPT-5.x, Gemini 3.x) für komplexe Refactorings, Architekturfragen und Agent-Tasks, gern kombiniert mit `/plan`. Und die **Mini-/Flash-Klasse** für viele kleine Massen-Tasks. Die Landschaft ändert sich laufend – koppele Prozesse deshalb nie an Modellnamen, sondern an Auto oder Modellklassen. `/model` zeigt immer den aktuellen Stand; mit `/model --repo` lässt sich ein Standard-Modell pro Repository setzen.

Budgets & Alerts – so fließt das Geld

Jede Lizenz zahlt ihr Guthaben in den gemeinsamen Pool ein; alle schöpfen zuerst daraus. Erst danach greift das Budget: Es erlaubt und begrenzt Zusatzverbrauch (gesetzt in USD; 10 \$ = 1.000 Credits), auf drei Ebenen (Enterprise → Organisation → User, universell oder individuell) plus Cost Center für Teams. Warn-Mails gehen automatisch bei 75 / 90 / 100 % an die Billing-Admins. Budget erreicht heißt Stopp: Betroffene sind blockiert bis Aufstockung oder neuer Zyklus. Die Billing-UI trennt inkludierte und zusätzliche Ausgaben, auch mitten im Zyklus. Neu seit Juli: Cost-Center-Deckel auch für den inkludierten Pool (zunächst per REST-API) und Credit-Limits pro Session direkt in CLI und SDK.

Selbsttest: Verbraucht das AI Credits? (Auflösung vom Quiz)

- 1 · **Nein** Inline-Completion beim Tippen – Flatrate.
- 2 · **Ja** Frage im Copilot Chat.
- 3 · **Nein** Next Edit Suggestion annehmen – Flatrate.
- 4 · **Ja** Session in der Copilot CLI.
- 5 · **Ja** Task an den Coding Agent – plus Actions-Minuten.
- 6 · **Ja** Copilot Code Review am PR – plus Actions-Minuten.

Modul 4 · Copilot CLI – Deep-Dive

Was die CLI ist

Der Coding Agent in deinem Terminal. Sie liest Dateien, schreibt Code und führt Befehle aus – du bestätigst jeden Schritt. Sie nutzt dieselbe Agent-Basis („Harness“) wie der Coding Agent in der Cloud, hat GitHub eingebaut (Repos, Issues, PRs per natürlicher Sprache), funktioniert editor-unabhängig auch remote/SSH und ist über MCP-Server, Custom Agents und Skills erweiterbar.

Zwei Betriebsarten

Interaktiv (copilot im Projektordner, Dialog-Session, Kontext bleibt erhalten) und **programmatisch** mit `copilot -p` für Skripte, Aliasse und CI-Pipelines. Der entscheidende Unterschied: `-p` kann nicht nachfragen. Reine Lese-Operationen laufen automatisch; alles, was das System verändert oder komplexere Shell-Befehle braucht, muss vorab erlaubt werden – immer minimal, nie pauschal `--allow-all` außerhalb einer Sandbox.

```
copilot -p "Fasse die letzten 5 Commits zusammen" \
-s --allow-tool="shell(git:*)"

-s = saubere Ausgabe fuer Skripte
"shell(git:*)" erlaubt exakt git (inkl. Subkommandos)
```

Praxiswert aus dem Test: ohne Flag liefen 36 Sekunden Permission-denied-Schleifen auf. Kostenpunkt: 7,5 Credits für nichts. Freigaben zuerst.

Agentisch arbeiten

`/plan` zwingt die CLI, erst einen Plan vorzulegen – du liest, korrigierst, gibst frei; der Plan ist ein Vertrag. **Autopilot** (Shift+Tab) arbeitet selbstständig bis zum Ziel – mächtig, aber nur in sicheren Umgebungen. Und `/delegate` übergibt an den Cloud Coding Agent: Branch und Draft-PR entstehen im Hintergrund, während du weiterarbeitest.

Slash-Befehle im Überblick

<code>/model</code>	Modell wählen; <code>--repo</code> setzt den Repo-Standard
<code>/context</code>	Auslastung des Kontextfensters (Tokens, Puffer) – nicht der Inhalt
<code>/usage</code>	Credit-Verbrauch der Session
<code>/compact</code>	Verlauf eindampfen, spart Tokens
<code>/plan</code>	erst Plan prüfen, dann umsetzen lassen
<code>/delegate</code>	Task an den Cloud Coding Agent → Draft-PR
<code>/review</code>	Code-Review anfordern
<code>/agent</code>	Agent auswählen: <code>/agent [name]</code>
<code>/agents</code>	Default- und Subagent-Modelle konfigurieren
<code>/env</code>	geladene Umgebung: Instructions, MCP, Skills, Agents, Hooks, Plugins
<code>/fleet</code>	Fleet-Modus – Subagents parallel ausführen
<code>/mcp · /plugins</code>	MCP-Server bzw. Plugins verwalten
<code>/sandbox enable</code>	Sandbox für die Session (Public Preview seit 06/2026)
<code>/rubber-duck</code>	adversariales Feedback, die Quietscheente
<code>/login · /help</code>	Anmeldung · alle Befehle

Die vollständige Liste inkl. `/clear`, `/cwd`, `/resume`, `/diff` findest du im Cheat-Sheet. Achtung: `/agent` (Agent wählen) und `/agents` (Subagent-Modelle) sind zwei verschiedene Befehle.

Neu: die Tab-Oberfläche

Seit Kurzem startet die CLI standardmäßig mit vier Ansichten: **Session** (die gewohnte Arbeit), **Issues**, **Pull requests** und **Gists** – dein GitHub-Überblick, ohne die CLI zu verlassen. Navigation mit der Pfeiltaste → (steht im Footer). Praktisch: Aus dem PR-Tab lässt sich direkt ein Git-Worktree zum Pull Request anlegen, Review-Checkout in Sekunden; #123-Verweise im Chat verlinken automatisch aufs aktuelle Repo.

Sicherheit im Terminal – die Regeln des Tages

- **Bestätigen statt blind ausführen:** jede Änderung lesen, verstehen, genehmigen.
- **Ordner-Trust bewusst vergeben** – die CLI arbeitet nur, wo du es erlaubst.
- **Keine Secrets in Prompts**, auch nicht „nur zum Testen“.
- **Autopilot & Auto-Approve** nur in sicheren Umgebungen (Branch, Container).
- **Die Sandbox:** Datei-Änderungen und Netzwerkzugriffe laufen unter einer Policy (Einmal-Bypass nur nach Zustimmung), lokal wie in der Cloud, zentral erzwingbar.

Drei Schichten: Bestätigung + Ordner-Trust + Sandbox.

Profi-Tipps für den Alltag

Klein beauftragen (Fehlerraten steigen mit der Aufgabengröße). Vor jedem Agent-Lauf committen, dann ist Verwerfen gefahrlos. Erst Plan, dann Code. Denkzeit erzwingen („Erklär erst deinen Lösungsweg“). Session-Hygiene (`/compact`, neue Session pro Thema). Und: Testen lassen – „... und führ `mvn -q test` aus“ gehört in den Auftrag. Custom Agents & Skills sind der nächste Schritt: Spezialisten per Datei-Konfiguration, Skills verpacken Teamwissen wiederverwendbar. Vertiefung: der Folge-Workshop.

Fünf Rezepte zum Kopieren

```
copilot -p "Fasse die letzten 5 Commits als Bullet-Liste" \
-s --allow-tool="shell(git:*)" # Standup in 10 Sek.

copilot -p "Commit-Message fuer staged Aenderungen \
(Conventional Commits)" -s --allow-tool="shell(git:*)"

copilot -p "Was hat sich zwischen main und meinem Branch \
geaendert?" -s --allow-tool="shell(git:*)"

copilot -p "Finde Auffaelligkeiten in build.log" -s
# Nur Lesen, kein Flag noetig: Reads sind automatisch erlaubt.

copilot -p "Release-Notes seit dem letzten Tag" \
-s --allow-tool="shell(git:*)" # CI-tauglich
```

Tipp: als Shell-Alias anlegen – zehn Minuten Aufwand, täglicher Ertrag.

Modul 5 · Copilot in IntelliJ

Stand 2026: weitgehend gleichauf mit VS Code. Completions & Next Edit Suggestions, Chat, Inline-Chat & Inline-Agent, Agent Mode inkl. Custom-, Sub- und Plan-Agents, MCP-Support mit Auto-Approve-Regeln, die Copilot-CLI-Integration mit Unified-Sessions-Ansicht (deine CLI-Sessions tauchen in der IDE mit auf), und seit 06/2026 Copilot als Agent im JetBrains AI Assistant. Setup: Plugin „GitHub Copilot“ installieren, OAuth-Login; für Preview-Features muss die Editor-Preview-Policy der Organisation aktiv sein. **Faustregel:** IDE zum Tippen und Navigieren, CLI zum Beauftragen und Delegieren – gleiche Lizenz, gleiche Regeln, AGENTS.md wirkt überall.

Modul 6 · Agents – der Ausblick

Drei Stufen agentischen Arbeitens

Der **Agent Mode** in der IDE ist der Praktikant neben dir – mehrschrittig, lokal, du bestätigst. Die **CLI** ist der Kollege am selben Tisch – gleiche Agent-Basis im Terminal. Der **Coding Agent** ist das Teammitglied im Homeoffice: Du briefst (per /delegate oder Issue), er arbeitet asynchron auf GitHub-Infrastruktur und liefert einen Draft-PR, parallelisierbar über mehrere Aufgaben.

Agent HQ („Mission Control“, GA)

Ein Ort für alle Agent-Arbeit, als Agents-Tab auf github.com, über Repos hinweg. Status im Blick (läuft · wartet auf Review · fertig), Berechtigungsstufen pro Agent, verschachtelte Subagents für Teilaufgaben. Starten kannst du überall: Terminal (/delegate oder gh agent-task create), IDE, github.com, mobil – und für Wiederkehrendes gibt es GitHub Agentic Workflows.

Die GitHub Copilot App (GA seit 17.06.)

Der Desktop fürs Delegieren: macOS, Windows, Linux. Die My-Work-Sicht bündelt Sessions, Issues, PRs und Automations; jede Session läuft in ihrem eigenen Git-Worktree (parallel, kollisionsfrei), und deine CLI-Sessions erscheinen dort mit. Die Highlights: **Canvases**, gemeinsame Arbeitsflächen, auf denen der Agent aktualisiert und du ordnest, korrigierst, genehmigst. **Agent Merge** begleitet den PR durch CI und Review (Branch Protection gilt unverändert). **Cloud Automations** erledigen geplante Agent-Arbeit, auch bei zugeklapptem Laptop. Dazu freie Modellwahl (BYOM) und MCP. Der Zugang läuft über dieselbe Org-Policy, die auch die CLI freischaltet.

Vom Terminal zum Pull Request – der Ablauf aus der Demo

1) /delegate mit klaren Akzeptanzkriterien (inklusive Testlauf im Auftrag). 2) Im Agent HQ oder der App den Status und die Zwischenschritte verfolgen. 3) Den Draft-PR prüfen: Diff, Begründung, hat er wirklich getestet? 4) Review wie immer: Checks, Kommentar, Merge-Entscheidung bleiben bei dir. Gute Beobachtungsfragen für jeden Agent-Lauf: Formuliert er Annahmen? Testet er selbst? Wie begründet er Änderungen?

Einsatzfelder heute

Klar umrissene, testbare Aufgaben: der Bugfix mit Reproduktionsschritten, kleine Features, Doku, Testlücken. **Noch nicht:** vage Anforderungen, große Architektur-Umbauten, sicherheitskritische Pfade ohne enge Review. **Die Leitplanken:** Ein Agent-PR ist ein normaler PR (Review-Pflicht und Branch Protection gelten), minimale Berechtigungen, Budgets im Blick (Credits + Actions-Minuten), volle Nachvollziehbarkeit über Sessions, Logs und Audit. Für die Governance: Policies werden zentral auf Org-Ebene gesetzt, Content Exclusions halten sensible Pfade aus dem Kontext, Audit-Logs machen alles nachvollziehbar, und die Filter aus Modul 1 laufen immer.

Woran ihr merkt, dass es wirkt

Was ihr messt: Adoption (aktive Nutzer je Lizenz, Nutzung, quantitativ über die GitHub-APIs), Downstream-Wirkung (Durchlaufzeit, PR-Frequenz, Review-Zeiten), qualitativ kurze Entwickler-Umfragen (Zahlen erklären nicht das Warum) und sichtbar gemacht in Dashboards. **So startet ihr:** Baseline vor dem Rollout, ein North-Star-Ziel statt zehn KPIs, das Engineering System Success Playbook (ESSP) als Rahmen, quartalsweise auswerten und nachsteuern – Adoption ist eine Reise.

FAQ – die Klassiker

Sind unsere Daten sicher?	Business/Enterprise: kein Training auf eurem Code, verschlüsselte Übertragung, Org-Policies steuern den Rest. Belastbare Quelle: das Copilot Trust Center.
Was, wenn die Credits alle sind?	Kein Fallback-Modell mehr, Budgets und Alerts durch Admins; Aufstockung statt Überraschung. Betroffene sind blockiert, bis aufgestockt wird.
Welchen Kontext nutzt ein Vorschlag?	IDE: aktive Datei plus Tabs. CLI/Agent: freigegebenes Verzeichnis, genannte Pfade und AGENTS.md.
Was sind Halluzinationen?	Plausible, aber falsche Ausgaben (APIs, Parameter), systembedingt, kein Defekt. Die Antwort ist nicht „auf ein besseres Modell warten“, sondern: Kontext geben, Referenzen mitgeben, Ergebnis testen lassen. Darum: Review-Pflicht.
Können wir Copilot einschränken?	Ja – Org-Policies, Content Exclusions und Berechtigungsstufen pro Agent.
Passt das in unsere Pipelines?	Ja – <code>copilot -p</code> in Skripten und CI (mit vorab erlaubten Tools), der Coding Agent per Issue, Code Review am PR.

Deine Trainer

Ahmed Diab & Daniel Snell · a.diab@reply.de & d.snell@reply.de

Cluster Reply, Agentic DevOps. Bei Fragen in den nächsten Wochen: einfach melden, wir antworten.